

Mikrocomputertechnik 1

Einheit 7: Performance & Pipelining

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 85

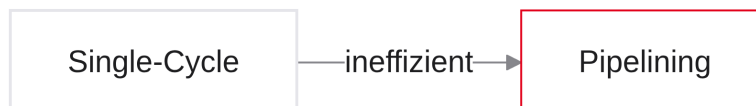
Performance & Pipelining

Folie

2 / 85

Fließbandarbeit im Prozessor

- Herzlich willkommen zur siebten Vorlesung in Mikrocomputertechnik 1
- Bisher: funktionierender, aber architektonisch langsamer Single-Cycle-Prozessor
- Heute: Fließbandarbeit (Pipelining) als Architektur-Revolution
- Mehrere Befehle gleichzeitig im Datenpfad
- Stolpersteine: Data Hazards und Control Hazards

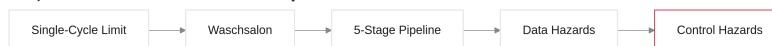


Notizen

Willkommen zu Einheit 7. Steuerwerk und Datenpfad stehen — der Single-Cycle-Prozessor ist logisch klar, aber langsam. Heute: Pipelining, Leistungsgewinn und Hazards; am Ende Ripes.

Agenda für den heutigen Tag

1. Performance-Grenzen — warum Single-Cycle eine Sackgasse ist
2. Die 5-stufige RISC-V-Pipeline — Datenpfad zersägen, Pipeline-Register
3. Data Hazards & Forwarding — Variablen-Abhängigkeiten (RAW)
4. Load-Use Hazards & Stalls — wenn Forwarding nicht reicht
5. Control Hazards — Sprünge, Flushes, Branch Prediction
6. Praxis (Ripes) — Hazards und Pipeline-Bubbles live



Notizen

Von der Performance-Analyse über die Waschsalon-Metapher zur fünfstufigen Pipeline. Danach Hazards und Ripes.

Rückblick: Der Single-Cycle-Datenpfad

- Bisher: jeder Befehl braucht exakt einen Taktzyklus
- Phasen Fetch, Decode, Execute, Memory, Write-Back aneinandergereiht
- Steuerwerk dirigiert fehlerfrei
- Logik korrekt — Silizium-Nutzung miserabel
- Warum verwerfen Apple, Intel und AMD dieses Design?

Ein Design ist nur dann gut, wenn es nicht nur logisch korrekt, sondern auch effizient ist.

Notizen

Der Prozessor funktioniert. Warum zersägen wir ihn? Weil er ineffizient ist. Silizium kostet; ungenutzte Hardware hat auf dem Markt keine Chance.

Die Timing-Falle (Refresher)

- Taktfrequenz diktiert vom kritischen Pfad
- Ein Takt muss das langsamste Gatter im schlechtesten Fall abdecken
- Langsamster Befehl: `lw` — nutzt alle Komponenten und setzt die Uhr
- Fazit: extrem langer Takt drosselt die gesamte CPU

```
Single-Cycle-Takt (kritischer Pfad):
Load Word: |=====| (gibt den Takt vor)
Add:       |=====|..... (wartet am Ende)
           ^ fertig    ^ verschwendete Zeit
```

Notizen

Aus Einheit 5: Timing-Falle. Ein Takt muss lang genug für den Speicherzugriff eines Load sein — die Uhr tickt langsam.

Ressourcen-Verschwendung

- Was passiert bei einem schnellen Befehl wie `add`?
- ALU und Write-Back sind nach einem Bruchteil des Takts fertig
- Fetch/Decode/ALU liegen den Rest des Zyklus still
- Massive Rechenleistung geht durch Warten auf die nächste Flanke verloren

Notizen

Bei `add` ist die ALU nach wenigen Nanosekunden fertig — dann Schlaf bis zum Taktende. Absolute Ressourcenverschwendung.

Wie steigert man Performance?

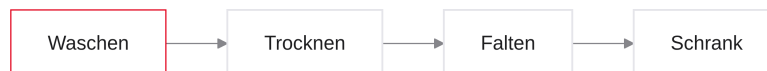
- Unbestechliche Formel der Rechnerarchitektur:
- CPU-Zeit = Befehlsanzahl · CPI · Taktzeit
- Befehlsanzahl: Codevolumen (Compiler-Optimierung)
- CPI (Cycles per Instruction): mittlere Takte pro Befehl
- Taktzeit: Dauer eines Clock-Ticks (Nanosekunden)

Notizen

Single-Cycle: CPI = 1, aber miserable Taktzeit. Ziel heute: Taktzeit drastisch senken, ohne den CPI völlig zu ruinieren.

Die Waschsalon-Metapher

- Pipelining intuitiv: kurz weg von Hardware, rein in den Waschsalon
- Vier Körbe Schmutzwäsche, vier Schritte je 30 Minuten:
- Waschen → Trocknen → Falten → Schrank
- Eine komplette Ladung dauert immer exakt 2 Stunden



Notizen

Pipelining ist kein Hexenwerk. Vier Stationen à 30 Minuten: ein Korb braucht zwei Stunden von Anfang bis Ende.

Sequenzielle Arbeit (Single-Cycle)

- Wie ein Single-Cycle-Prozessor wäscht:
- Korb 1 komplett durch alle Stationen
- Erst wenn Korb 1 im Schrank liegt, startet Korb 2
- Vier Körbe: $4 \times 2 \text{ h} = 8 \text{ h}$
- Die Waschmaschine steht fast die ganze Zeit still

```
Timeline sequenziell:
[ Korb 1: Waschen -> Trocknen -> Falten -> Schrank ]
|<----- 2 Stunden ----->|
^ erst hier startet Korb 2
```

Notizen

Sequenziell: Maschine läuft 30 Minuten, dann 90 Minuten Pause. Vier Körbe brauchen acht Stunden — teure Hardware langweilt sich.

Folie

10/85

Pipelined Waschen (Fließband)

- Intelligente Variante: Fließband (Pipeline)
- Sobald Korb 1 in den Trockner wandert, geht Korb 2 in die Waschmaschine
- Alle vier Stationen arbeiten gleichzeitig an vier Körben
- Vier Körbe: nur noch 3,5 Stunden (statt 8)

```
Pipelined (vereinfacht):
Zeit:      0.5h   1.0h   1.5h   2.0h   2.5h   3.0h   3.5h
Korb 1:    Wash  Dry    Fold   Shelf
Korb 2:           Wash  Dry    Fold   Shelf
Korb 3:                Wash  Dry    Fold   Shelf
Korb 4:                        Wash  Dry    Fold   Shelf
```

Notizen

Entkopplung: freie Station sofort neu beladen. Parallelität spart fast fünf Stunden bei vier Körben.

Folie

11/85

Latenz vs. Durchsatz

- Häufigster Denkfehler beim Pipelining
- Hat Pipelining die Latenz eines einzelnen Korbs verkürzt?
- Nein: Korb 1 braucht weiterhin exakt 2 Stunden
- Pipelining wäscht nicht heißer oder schneller
- Pipelining erhöht ausschließlich den Durchsatz (Throughput)

```
Latenz      = Wie lange dauert EIN Befehl von Anfang bis Ende?
Durchsatz   = Wie viele Befehle werden pro Sekunde fertiggestellt?
```

Notizen

Prägen Sie sich ein: Latenz eines Tasks bleibt; der Output der Anlage steigt. Karriere-relevant in der Informatik.

Folie

12 / 85

Der Durchsatz (Throughput)

- Magie des Pipelining: Taktrate des Outputs
- Sequenziell: alle 2 Stunden ein fertiger Korb
- Pipeline (voll beladen): alle 30 Minuten ein fertiger Korb
- Durchsatz vervierfacht
- Genau so werden Computer und Grafik flüssig

Notizen

Sobald das Fließband warmgelaufen ist, explodiert der Output. Videospiele: Millionen Pixel pro Sekunde — reiner Durchsatz.

Folie

13 / 85

Transfer auf den Prozessor

- Wäschestationen → fünf RISC-V-Ausführungsphasen
- IF: Instruction Fetch (Befehl aus dem IM)
- ID: Instruction Decode (Decoder & Register File)
- EX: Execute (ALU)
- MEM: Memory (Daten-RAM)
- WB: Write-Back (Ergebnis ins Register)



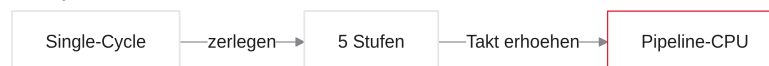
Notizen

Bisher: ein Befehl durch alle fünf Stationen, dann der nächste. Ab sofort: ALU an A, Decoder an B, Fetch an C — parallel.

Die 5-stufige RISC-V-Pipeline

Den Prozessor physisch umbauen

- Säge ansetzen: Datenpfad in Stufen zerlegen
- Schnellerer Takt rettet die Performance
- Isolations-Problem: Signal-Chaos ohne Barrieren
- Pipeline-Register als Schotten
- Idealer Speedup: Faktor 5



Notizen

Phase 2: Theorie in Hardware. Single-Cycle-Datenpfad aus Einheit 5 in fünf Inseln schneiden und Takt beschleunigen.

Den Datenpfad zersägen

- Vertikale Schnitte durch den Patterson/Hennessy-Schaltplan
- Fünf Hardware-Inseln:
- IF: PC und Befehlsspeicher
- ID: Register File und Control Unit
- EX: Haupt-ALU
- MEM: Daten-RAM
- WB: Write-Back-MUX vor dem Register File

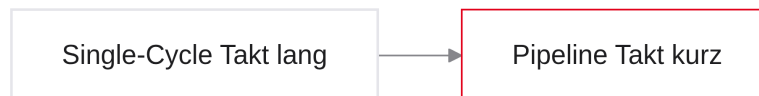
[IF]		[ID]		[EX]		[MEM]		[WB]
Stufe 1		Stufe 2		Stufe 3		Stufe 4		Stufe 5

Notizen

Keine Gatterlogik ändern: Strom fließt weiter von links nach rechts. Nur die Strecke wird in fünf Abschnitte geteilt.

Ein viel schnellerer Takt

- Clock muss nicht mehr die Summe aller fünf Stufen abdecken
- Nur noch die langsamste Einzelstufe (oft ALU oder RAM)
- Jede Stufe = Bruchteil der Gatter → Takt massiv verkürzbar
- Beispiel: 4 GHz statt 800 MHz (Größenordnung)



Notizen

Kritischer Pfad: vorher durch alle Inseln, jetzt durch die längste. Taktfrequenz steigt stark — neues physikalisches Problem folgt.

Das Isolations-Problem

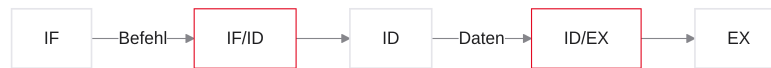
- Ohne Schutz: Signal-Chaos nach dem Zersägen
- Kombinatorik fließt ungebrems weiter
- ALU-Ergebnis rutscht in MEM, bevor der nächste Tick kommt
- Befehle krachen ineinander (Auffahrunfall)
- Dicke Wände zwischen den Inseln nötig

Notizen

Gatter sind offene Rohre. Ohne Barrieren vermischt sich der Fließband-Verkehr — Inseln müssen elektrisch isoliert werden.

Die Pipeline-Register (Barrieren)

- Lösung: Pipeline-Register (Flip-Flop-Bänke)
- Wasserdichte Schotten an den Grenzen der fünf Inseln
- Speichern Zwischenergebnisse der vorherigen Stufe
- Namen: IF/ID, ID/EX, EX/MEM, MEM/WB



Notizen

D-Flip-Flop-Bänke wie Luftschleusen: IF/ID hält das 32-Bit-Wort, bis ID bereit ist. Strom fließt nicht mehr unkontrolliert.

Takt 1: Fetch

- Start des Fließbands im Timing-Raster
- Instruktion 1 in IF
- PC adressiert, IM liefert den Befehl
- Befehl staut sich am IF/ID-Register
- ID, EX, MEM, WB noch leer

Takt 1:

```
[ I1 IF ] -> [ ID - ] -> [ EX - ] -> [ MEM - ] -> [ WB - ]
```

Notizen

Takt 1: Instruktion 1 prallt gegen die Flip-Flop-Mauer IF/ID. Stufen 2–5 schlafen noch.

Takt 2: Fetch & Decode

- Nächster Clock-Tick öffnet kurz die Schleusen
- Instruktion 1 rutscht nach ID (Decode, Control Unit)
- Instruktion 2 betritt IF
- Zwei Inseln arbeiten parallel an zwei Instruktionen

```
Takt 2:  
[ I2 IF ] -> [ I1 ID ] -> [ EX - ] -> [ MEM - ] -> [ WB - ]
```

Notizen

Echte Parallelität: Decode analysiert Instruktion 1, Fetch liest Instruktion 2.

Takt 5: Die volle Pipeline

- Fließband auf Betriebstemperatur
- Alle fünf Stufen lückenlos gefüllt
- Instruktionen 5–1 in IF–WB
- Ab diesem Takt: bei jedem Tick ein fertiger Befehl am Ende

```
Takt 5 (volles Fließband):  
[ Instr 5 (IF) ] -> [ Instr 4 (ID) ] -> [ Instr 3 (EX) ]  
-> [ Instr 2 (MEM) ] -> [ Instr 1 (WB) ]
```

Notizen

Maximale Last: Instruktion 1 schreibt zurück, während Instruktion 5 gerade geholt wird. Hochleistungsfabrik.

Das Pipeline-Diagramm

- Standardwerkzeug für komplexe Abläufe
- X-Achse: Taktzyklen (Zeit)
- Y-Achse: Programm-Instruktionen (oben nach unten)
- Diagonal: Befehle wandern durch die Stufen

Befehl	1	2	3	4	5	6	7
add	IF	ID	EX	MEM	WB		
sub		IF	ID	EX	MEM	WB	
lw			IF	ID	EX	MEM	WB

Notizen

Klassisches Klausur-/Lehrbuchdiagramm: Treppenstruktur als Sinnbild des Pipelining.

Folie

24 / 85

Der ideale Speedup

- Theoretisch: Speedup = Anzahl der Stufen
- Fünf Stufen → idealer Speedup 5
- Beispiel: Programm, das single-cycle 5 Minuten braucht → ideal 1 Minute
- Taktzeit bricht stark ein; Durchsatz steigt

Ideale Belohnung:
CPI bleibt ~1, Taktzeit sinkt drastisch
→ Durchsatz steigt mit der Stufenzahl

Notizen

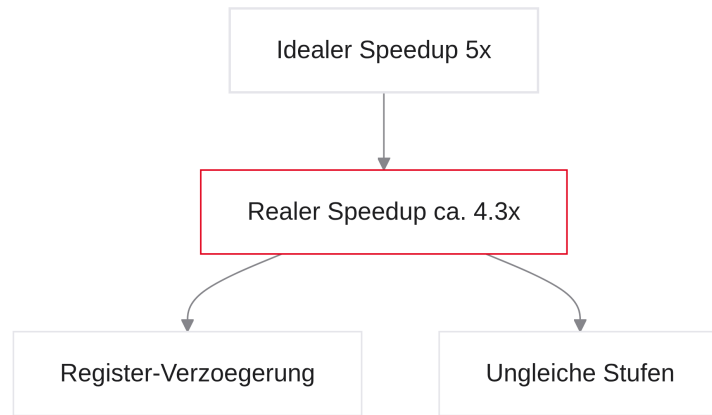
Traum des Ingenieurs: idealer Speedup gleich Stufenzahl (bei vollem Fließband und ohne Hazards). CPI bleibt ideal ~1, aber mit viel kürzerer Taktzeit. Hazards und Overhead senken den realen Faktor.

Folie

25 / 85

Warum nicht genau fünffach?

- Real eher Faktor 4 bis 4,5
- Overhead: Pipeline-Register kosten Setup-Zeit
- Imbalance: Stufen nie exakt gleich lang (RAM oft langsamer als ALU)
- Uhr richtet sich nach der längsten Stufe
- Trotzdem: eine der wichtigsten Performance-Technologien



Notizen

Hardware-Overhead und ungleiche Stufen drosseln den Idealwert — der Gewinn bleibt gigantisch.

Folie

26 / 85

Steuersignale in der Pipeline

- Was wird aus der Control Unit (Einheit 6)?
- Steuerwerk sitzt in ID und liest den Opcode
- Beispiel: `MemWrite = 1` für Store
- Problem: Schreiben erst zwei Takte später in MEM
- Lösung: Steuersignale reisen Huckepack mit dem Befehl

Notizen

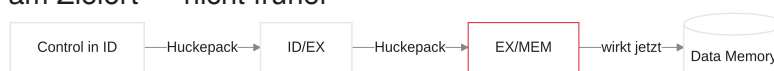
Zwischen ID und MEM liegen zwei Takte. Ohne Mitnahme wäre `MemWrite` in MEM längst vergessen.

Folie

27 / 85

Das Wandern der blauen Kabel

- Steuersignale in die Pipeline-Register packen
- `MemWrite` wandert ID → ID/EX → EX/MEM
- Erst wenn der Befehl in MEM steht, wirken die Signale am RAM
- Aktion erst am Zielort — nicht früher



Notizen

Blaue Leitungen durch die Barrieren: Pipeline-Register speichern Daten und Steuersignale. MemWrite als blinder Passagier bis zum RAM.

Folie

28 / 85

Die perfekte Illusion

- Bisher: Hardware-Utopie — CPI ~ 1, Takt auf Höchststand
- Fatale Annahme: alle Befehle seien unabhängig
- Realität: Code-Zeilen brauchen ständig Daten voneinander
- Nächster Schritt: die Gefahren (Hazards) des Fließbands

```
/* Illusion: alles unabhaengig */  
int a = 5;  
int b = 6;  
int c = 7;
```

Notizen

Pipeline funktioniert nur perfekt bei unabhängigen Befehlen. Software ist verflochten — als Nächstes: Blockaden und Hazards.

Folie

29 / 85

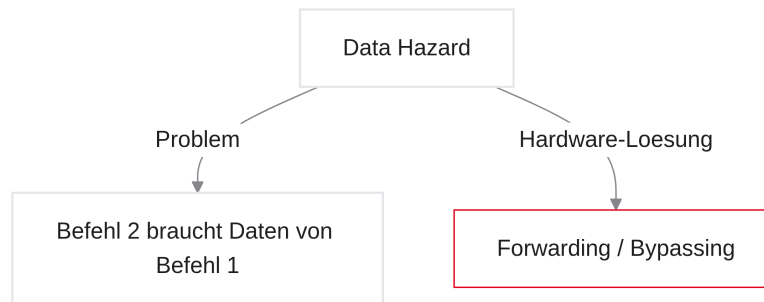
Data Hazards & Forwarding

Folie

30 / 85

Utopie beenden

- Wir betrachten echten, abhängigen Software-Code
- Definition von Hazards (Gefahren auf dem Fließband)
- Problem: Read-After-Write (RAW)
- Warum Stalling die Performance wieder vernichtet
- Lösung: Forwarding (Bypassing) als Abkürzung



Notizen

Phase 3: Befehle sind in der Realität abhängig. Ein Befehl berechnet eine Variable, der nächste nutzt sie sofort. Auf dem Fließband entstehen Data Hazards — wir retten die Pipeline davor.

Folie

31 / 85

Was ist ein Hazard?

- Situation, die verhindert, dass der nächste Befehl planmäßig im nächsten Takt starten kann
- Das Fließband muss stottern oder korrigieren
- Drei Kategorien:
- Structural Hazards: zwei Befehle wollen dieselbe Hardware im selben Takt
- Data Hazards: Abhängigkeit von noch nicht fertigen Daten
- Control Hazards: Sprung ändert den PC, während schon falscher Code geladen ist

Notizen

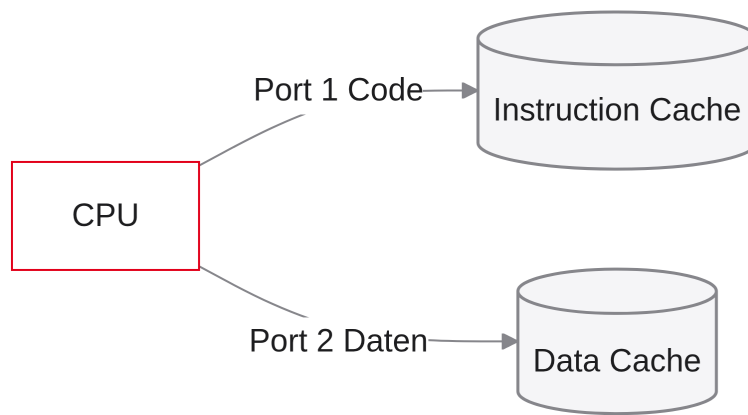
Hazard = Fließband-Gefahr. Heute praxisrelevant: Data Hazards und Control Hazards. Structural Hazards klären wir kurz vorab.

Folie

32 / 85

Structural Hazards (Ressourcenkonflikte)

- Befehl in MEM will den RAM schreiben, Befehl in IF will Code lesen
- Gleicher Takt, ein Adress-Port — physikalischer Konflikt
- Lösung: getrennte I/D-Speicher (modified Harvard)
- Instruction Memory und Data Memory (bzw. getrennte L1-Caches)
- In unserem Modell treten Structural Hazards daher praktisch nicht auf



Notizen

Ein gemeinsamer Speicher für Code und Daten würde IF und MEM kollidieren lassen. Im Lehrbuch-/Ripes-Modell (wie in Einheit 5) trennen wir Instruction Memory und Data Memory bzw. getrennte L1-I/D-Caches — oft als modified Harvard bezeichnet. Die RISC-V-ISA schreibt Harvard nicht vor; die getrennten Ports lösen hier den Structural Hazard.

Folie

33 / 85

Data Hazards (Das Hauptproblem)

- Entstehen durch logische Abhängigkeiten im Quellcode
- Alltag in C: $a = b + c$; dann $d = a + e$;
- d braucht das fertige a
- Single-Cycle: kein Problem — Zeile 1 ist fertig, bevor Zeile 2 startet
- Pipeline: Überhol-Problem auf dem Fließband

```
/* Klassischer Data Hazard (RAW): */  
int a = 10 + 5;    /* schreibt a */  
int d = a + 20;    /* liest a sofort danach */
```

Notizen

Im Single-Cycle lag das Ergebnis fest im Register File, bevor der nächste Befehl startete. In der Pipeline überholt der Leser den Schreiber.

Read After Write (RAW) in Assembler

- C-Snippet als RISC-V: add gefolgt von sub
- add t0, t1, t2 speichert in t0
- sub t3, t0, t4 liest t0 sofort
- Konflikt: sub verlangt einen Wert, der noch nicht im Register File liegt
- Schreiben erst in WB (Stufe 5), Lesen schon in ID (Stufe 2)

```
add t0, t1, t2    # t0 erst in WB (Stufe 5)
sub t3, t0, t4    # t0 schon in ID (Stufe 2) gelesen
```

Notizen

RAW: Write von add, Read von sub. Physikalisch klaffen Stufe 5 und Stufe 2 auseinander.

Kollision auf dem Fließband

- add schreibt t0 erst in Takt 5 (WB)
- sub liest t0 schon in Takt 3 (ID)
- Katastrophe: sub liest veraltete Daten — falsche Rechnung

Befehl	1	2	3	4	5	6
add t0, t1, t2	IF	ID	EX	MEM	WB	
sub t3, t0, t4		IF	ID	EX	MEM	WB

Notizen

In Takt 3 ist sub in ID und liest t0, während add erst in EX ist. Das Fließband produziert Müll.

Die naive Lösung: Stalling

- Hardware erkennt die Abhängigkeit und hält an (Stall)
- `sub` bleibt in ID, bis `add` WB abgeschlossen hat
- Preis: typisch drei verlorene Taktzyklen (Konvention ohne Write/Read-Split im selben Takt; mit H&H-Registerfile-Hälften oft nur 2 Bubbles)
- Der Speedup durch kürzeren Takt wird teilweise zunichtegemacht

```
ADD:  IF ID EX MEM WB(schreibt)
SUB:      IF .. .. ID(liest sicher) EX MEM WB
           ^--- drei Takte Warten ---^
```

Notizen

Stall rettet die Korrektheit, vernichtet aber Durchsatz. Gibt es einen besseren Weg?

Die schlaue Lösung: Forwarding

- Wann ist das `add`-Ergebnis wirklich fertig?
- Nicht erst in WB — die ALU hat es am Ende von EX (Takt 3)
- WB ist nur der bürokratische Speichervorgang
- Idee: Ergebnis direkt am ALU-Ausgang abgreifen und zur nächsten ALU leiten
- Kein Umweg über das Register File

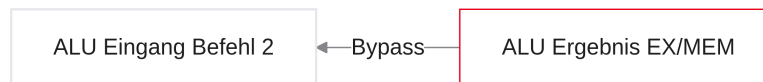
```
Forwarding (Bypassing):
Nutze das Ergebnis, sobald es physikalisch fertig ist.
Warte nicht auf Write-Back.
```

Notizen

Am Ende von EX liegt die Summe im EX/MEM-Register bereit. Wir reichen sie dem bedürftigen Befehl über den Tisch.

Bypassing (Die Abkürzung)

- Forwarding = Weiterleiten; Bypassing = Umgehen
- Neue Kupferkabel vom EX/MEM-Ausgang zurück an den ALU-Eingang
- MUX wählt die Abkürzung statt des (falschen) Register-Werts
- Konflikt gelöst, ohne einen Takt zu pausieren



Notizen

Gegen die normale Flussrichtung: frisches Ergebnis aus dem Bypass-Kabel in die Recheneinheit, statt Müll aus dem Register File.

Forwarding-Pfade im Datenpfad

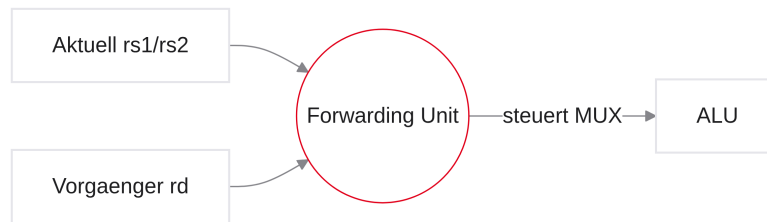
- Abkürzungen von allen Stellen, an denen Daten fertig werden
- Pfad 1: EX/MEM → ALU (direkt aufeinanderfolgende Befehle)
- Pfad 2: MEM/WB → ALU (ein unabhängiger Befehl dazwischen)
- Neue Stränge kreuzen das Diagramm von rechts nach links

Notizen

Zwei kritische Bypässe: direkt nach der ALU und weiter hinten vor dem Write-Back.

Die Forwarding Unit

- Neues Logiknetzwerk vor der EX-Stufe
- Vergleicht Registernummern der Befehle auf dem Fließband
- Frage: Ist `rd` des Vorgängers gleich `rs1/rs2` des aktuellen Befehls?
- Wenn ja: MUX auf die Abkürzung schalten



Notizen

Verkehrsbeobachter: Ziel des einen und Quelle des anderen vergleichen — bei Treffer Bypass-Alarm.

Folie

41 / 85

Die Hazard-Erkennung (Logik)

- Formale Bedingung (vereinfacht):
- Wenn $EX/MEM.RegWrite = 1 \wedge EX/MEM.rd \neq 0 \wedge EX/MEM.rd = ID/EX.rs1$
- Übersetzt: Vorgänger speichert, Ziel ist nicht x0, Ziel = meine Quelle
- Dann: ForwardA-MUX auf Bypass (z. B. 10)

```

if (vorheriger_schreibt && sein_ziel == meine_quelle) {
    aktiviere_abkuerzung();
}
  
```

Notizen

Großes AND: RegWrite, nicht x0, rd == rs1. Dann Weiche umlegen. Analog für rs2.

Folie

42 / 85

MUX-Erweiterung an der ALU

- Die 2-zu-1-MUX aus Einheit 5 reichen nicht mehr
- Upgrade auf 3-zu-1-Multiplexer
- Eingang 00: Register File / ImmGen
- Eingang 10: Bypass aus EX/MEM (ALU-frisch)
- Eingang 01: Bypass aus MEM/WB
- Forwarding Unit steuert die Herkunft (2-Bit-Steuersignal)

Notizen

Drei Pfade, 2-Bit-Steuerung. Die ALU weiß nicht mehr nur „Register oder Immediate“, sondern auch

Trace: Data Hazard gelöst

- Mit Forwarding: Zeitreise per Bypass
- `sub` bekommt das ALU-Ergebnis rechtzeitig in seiner EX-Stufe
- Kein Taktverlust

Befehl	1	2	3	4	5	6
<code>add t0, t1, t2</code>	IF	ID	EX	MEM	WB	
<code>sub t3, t0, t4</code>		IF	ID	EX	MEM	WB

Notizen

`add` fertig in EX (Takt 3); in Takt 4 reicht Forwarding das Paket in die EX von `sub`. Hazard besiegt.

Keine Zeit verloren

- Forwarding rettet den Speedup der 5-Stufen-Pipeline
- Takt bleibt kurz; Software merkt nichts vom Trick
- Zusammenspiel aus Datenpfad-Erweiterung und Kontroll-Logik
- Aber: Ist Forwarding allmächtig?

Performance-Check:

Bei RAW von Rechenbefehlen: 0 Taktzyklen Verlust.

CPI bleibt ~1.0

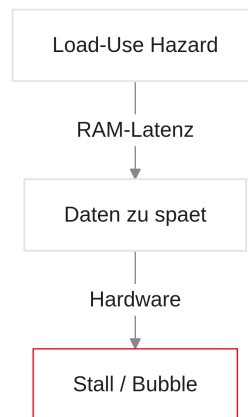
Notizen

Triumph bei ALU-zu-ALU-Abhängigkeiten. Es gibt jedoch einen Befehl, bei dem die Physik im Weg steht.

Load-Use Hazards & Stalls

Grenzen der Zeitreise

- Gefährlichster Datenkonflikt: Load-Use Hazard
- RAM-Latenz sabotiert das Fließband
- Zwangspause: Pipeline-Bubble (NOP)
- Rettung durch Compiler (Instruction Scheduling)



Notizen

Phase 4: Forwarding scheitert, wenn Daten erst aus dem RAM kommen. Stall und Bubble — und Hilfe vom Compiler.

Ist Forwarding unbesiegbar?

- Voraussetzung: Daten existieren schon irgendwo auf dem Chip
- Bei `add/sub`: Ergebnis am Ende von EX fertig
- Nächster Befehl braucht sie in EX — Timing passt
- Frage: Welcher Befehl liefert das Ergebnis erst nach EX?

Notizen

Bei `lw` rechnet die ALU nur die Adresse. Die Nutzdaten liegen noch im Arbeitsspeicher.

Der Load-Use Hazard

- Fataler Code: `lw` direkt gefolgt von Nutzung
- `lw t0, 0(s1)` dann `add t3, t0, t2`
- Daten eines `lw` erst am Ende von MEM (Stufe 4)
- Der RAM ist träge

```
lw  t0, 0(s1)    # Daten erst in MEM (Takt 4)
add t3, t0, t2    # braucht Daten schon in EX (Takt 4)
```

Notizen

In Takt 4: `lw` in MEM liefert erst am Taktende; `add` in EX braucht die Daten sofort.

Warum Forwarding hier versagt

- RAM-Daten erst am Ende von Takt 4 verfügbar
- `add` braucht sie am Anfang von Takt 4
- Ein voller Takt zu spät

Befehl	1	2	3	4	5	6
<code>lw t0, 0(s1)</code>	IF	ID	EX	MEM	WB	
<code>add t3, t0, t2</code>		IF	ID	EX	MEM	WB

Notizen

Paradoxon in Takt 4: Produzent fertig erst am Ende, Konsument braucht sie am Anfang.

Forwarding rückwärts in der Zeit?

- Bypass müsste vom MEM-Ausgang (Ende Takt 4) zum EX-Eingang (Anfang Takt 4)
- Das wäre Datenrücktransport in die Vergangenheit
- Physik verbietet es — harte Wand



Notizen

Strom fließt nur vorwärts in der Zeit. Die Daten existieren in diesem Moment noch nicht in der CPU.

Die Hardware-Lösung: Der Stall

- Hazard Detection Unit greift in der ID-Stufe ein
- `lw` fließt normal weiter nach MEM
- Abhängiger Befehl wird in ID eingefroren (kein Wechsel nach EX)
- PC ebenfalls eingefroren (kein neuer Fetch)

`Erzwungener STALL = Kaffeepause-Takt fuer das Fließband`

Notizen

Notbremse: Produzent darf Daten holen, Konsument und Fetch stehen still.

Was ist eine Pipeline-Bubble?

- Lücke im Fließband, weil der Folgebefehl auf der Stelle tritt
- EX ist einen Takt leer
- Hardware füllt mit einer Bubble: wirkungsloser NOP
- Bubble wandert EX → MEM → WB und tut nichts

Notizen

Luftblase statt Chaos: Die Stufe bekommt Arbeit, die aber Register und RAM nicht anfasst.

Hardware-Implementierung (NOP)

- Control Unit nullt die Erlaubnis-Pins
- Hazard Unit: `RegWrite = 0, MemWrite = 0, MemRead = 0`
- Befehl darf nichts Speichern/Schreiben
- Hardware „schläft“ einen Takt

```
NOP in Assembler (Beispiel):
addi x0, x0, 0
# sinnlos, ungefährlich, wirkungslos = Bubble
```

Notizen

NOP = alle sensiblen Steuersignale auf Null. ALU-Müll prallt am geschlossenen Register File ab.

Load-Use Timeline mit Stall

- add einen Takt in der Warteposition
- Bubble füllt die EX-Lücke
- Danach greift klassisches Forwarding (MEM → EX)

Befehl	1	2	3	4	5	6	7
<code>lw t0, 0(s1)</code>	IF	ID	EX	MEM	WB		
<code>add t3, t0, t2</code>		IF	ID	stall	EX	MEM	WB

Notizen

In Takt 5: `add` in EX, `lw` hat MEM beendet — Forwarding reicht die RAM-Daten rechtzeitig.

Der Preis des Stalls

- Logisch korrekt, Performance leidet
- CPI steigt (Bubbles verschwenden Takte)
- Viele Arrays laden + sofort verrechnen → viele Blasen
- Hardware am Limit — Rettung durch Software

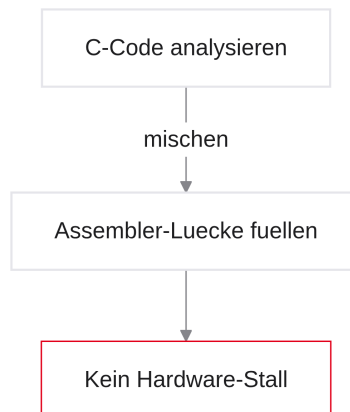
Jeder Load-Use Hazard: typisch 1 Takt Penalty

Notizen

Forwarding und Stalling sind ausgereizt. Der Compiler muss helfen.

Die Software-Lösung (Compiler)

- Variable nach lw nicht sofort im nächsten Befehl nutzen
- Compiler: Instruction Scheduling
- Unabhängige Instruktionen zwischen lw und Nutzung schieben



Notizen

Der Compiler kennt die Pipeline und verschiebt unabhängige Arbeit in die Hazard-Lücke.

Befehle umsortieren (Scheduling)

- Schlecht (1 Stall):
- `lw / add` (abhängig) / `sub` (unabhängig)
- Gut (0 Bubbles):
- `lw / sub` (Lücke füllen) / `add` (Daten haben Zeit)

```
# Bubble durch produktive Arbeit ersetzt:
lw  t0, 0(s1)
sub t4, t5, t6    # unabhaengig, fuehlt die Luecke
add t3, t0, t2    # Forwarding greift
```

Notizen

Dasselbe Programm, ein Takt schneller: Scheduling ersetzt die leere Bubble.

Die Macht des Compilers

- Generierter Assembler wirkt oft vermischt und unlogisch
- Absicht: Pipelines maximal auslasten
- Mit `-O3` (GCC) aggressive Umsortierungen
- Hardware und Software müssen Hand in Hand arbeiten

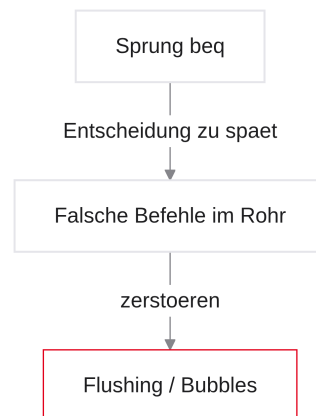
Notizen

Für Menschen Chaos, für die Pipeline ein choreografierter Tanz um RAM-Latenzen.

Control Hazards & Branch Prediction

Der Endgegner des Fließbands

- Was passiert, wenn der PC „lügt“?
- Falsche Fließband-Daten zerstören (Flushing)
- Performance-Einbruch durch Sprünge
- Moderne CPUs als Propheten: Branch Prediction



Notizen

Phase 5 beginnt: Data Hazards gemeistert. Control Hazards: `beq` wirft den Kontrollfluss um, während die Pipeline schon falschen Code geladen hat.

Der Control Hazard (`beq`)

- Data Hazards gemeistert — schwerster Konflikt: Kontrollfluss
- Beispiel: `beq t0, t1, ZielLabel`
- Single-Cycle: ALU in Takt 1, PC springt am Taktende
- Pipeline: Fließband steht niemals still
- Während `beq` durch die ersten Stufen kriecht, muss der PC in jedem Takt die nächste Fetch-Adresse wählen

Fundamentales Problem:

Der PC muss wissen, wo er weitergeht (Fetch), BEVOR die CPU verstanden hat, ob sie springen muss!

Notizen

Control Hazard: Der PC zieht stur den nächsten Befehl, ohne zu wissen, dass in der Röhre ein Sprung steckt, der die Route ändert.

Die Latenz der Entscheidung

- Wann fällt die endgültige Sprungentscheidung?
- Subtraktion $t0 - t1$ in der Haupt-ALU (Stufe EX, Einheit 5)
- Zero-Flag erst am Ende von Takt 3
- AND vor dem PC gibt die Zieladresse erst zu Beginn von Takt 4 frei

```
beq-Latenz:
Takt 1: IF
Takt 2: ID (Opcode bekannt, Bedingung noch unklar)
Takt 3: EX (ALU rechnet, Zero am Ende)
Takt 4: PC kann umschalten
```

Notizen

In ID weiß die CPU nur, dass es ein Branch ist — nicht, ob die Bedingung gilt. Die ALU in EX entscheidet.

Die Fließband-Falle

- Zwei Takte lang hat das Fließband blind weitergearbeitet
- Takt 2: Fetch bei $PC + 4$
- Takt 3: Fetch bei $PC + 8$
- Takt 4: Zero = 1 — eigentlich zu ZielLabel springen
- Die Befehle in IF und ID sind falsch

Befehl	1	2	3	4
beq $t0, t1, \text{Label}$	IF	ID	EX	MEM
Befehl bei $PC + 4$		IF	ID	EX
Befehl bei $PC + 8$			IF	ID

Notizen

Bei Branch Taken sitzen ahnungslose Folgebefehle in IF/ID — sie gehören nicht mehr zum Ablauf.

Branch Taken: Flushing

- Falsche Befehle in der Röhre müssen vernichtet werden
- Sonst: falsche Variablen, Absturz
- Flush: Steuersignale in IF und ID auf 0
- Falsche Befehle werden zu harmlosen Bubbles (NOPs)
- PC schwenkt auf Label; Fetch startet neu

Notizen

Flushing spült die Pipeline. System überlebt, Preis ist hoch.

Der Performance-Verlust (Penalty)

- Erfolgreicher Sprung: typisch 2 Takte Branch Penalty (ohne Optimierung)
- 15–20 % der Befehle sind Branches (If/Else, Schleifen, Calls)
- Ständiges Flushen zerstört den Throughput
- Nächster Schritt: Penalty in Hardware reduzieren

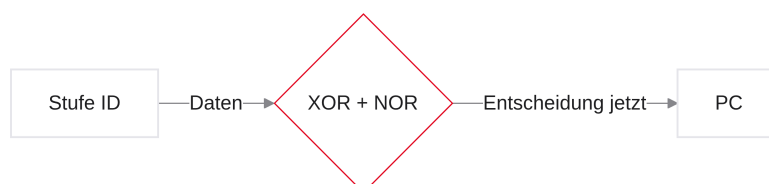
Ein Fließband ist nur schnell, wenn es nicht stoppt.
Jeder Flush ist ein Riss im Getriebe.

Notizen

Zwei Takte pro Sprung summieren sich: fast 30–40 % Verlust bei naivem Flushen.

Hardware-Optimierung: Der Komparator

- Branch-Logik nach links schieben (früher entscheiden)
- Extra-Komparator (XOR + NOR) in ID: Register vergleichen, sobald gelesen
- Entscheidung schon in Takt 2
- Penalty sinkt von 2 Takten auf 1 Takt



Notizen

Nicht auf die schwere ALU in EX warten: XOR + NOR in ID (Gleichheit, wenn alle Bit-XORs null sind). Ein Flush statt zwei — Lehrbuch-Standard zur Penalty-Reduktion. In realen CPUs ist Early Branch Resolve oft komplexer (eigene Vergleichslogik, Adressberechnung parallel, teils mit Prediction gekoppelt) — das XOR+NOR-Modell hier ist die didaktische Vereinfachung. Für tiefe Pipelines reicht selbst 1 Takt Penalty oft nicht; dort kommt Branch Prediction.

Folie

67 / 85

Die ultimative Rettung: Branch Prediction

- Moderne Pipelines: oft weit mehr als 5 Stufen (bis ~20)
- Flush würde Dutzende Takte kosten
- Lösung: Branch Prediction (Sprungvorhersage)
- Prozessor rät spekulativ, wohin es geht, und lädt Folge-Befehle

Notizen

Königsklasse: spekulativ weiterarbeiten, bevor die Bedingung bekannt ist.

Folie

68 / 85

Statische Vorhersage

- Simpleste Form (z. B. einfache Mikrocontroller)
- Assume Not Taken: immer $PC + 4$ nachladen
- Richtig geraten: null Taktverlust
- Falsch: Flush — im Schnitt oft noch Gewinn

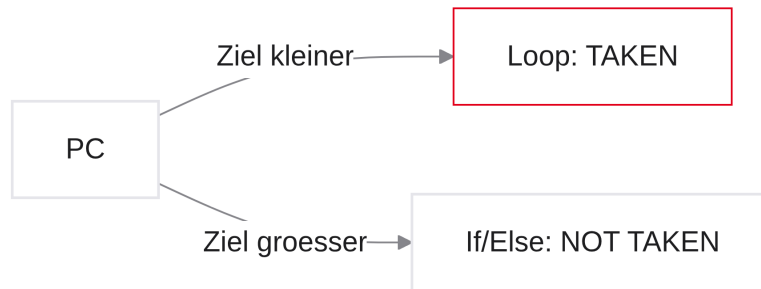
Assume Not Taken:
Nehmen Sie an, der Sprung werde nicht genommen,
bis der Zero-Ausgang des Vergleichers Sie eines Besseren belehrt.

Notizen

Naive, aber wirksame Strategie, wenn Sprünge selten genommen werden.

Schleifen-Vorhersage (Heuristik)

- Schleifen springen meist rückwärts (Taken)
- Schleifenende: einmal vorwärts raus (Not Taken)
- Heuristik:
 - Zieladresse $<$ PC $\rightarrow$ Prediction Taken (Schleife)
 - Zieladresse $>$ PC $\rightarrow$ Prediction Not Taken (If/Else)



Notizen

For-Schleife: 999× zurück, 1× raus — Rückwärtssprung als Taken macht die Schleife fast penalty-frei.

Dynamische Vorhersage (BHT)

- Dynamic Branch Prediction: lernen zur Laufzeit
- Branch History Table (BHT): Eintrag pro Sprungadresse
- Einfachstes Modell: 1 Bit (letzte Aktion = Prediction)
- Praxis/Vertiefung: oft 2-Bit-Saturating Counter
- Beim Wiedersehen derselben PC-Adresse: Prediction aus der Tabelle

PC-Adresse	Letzte Aktion	Prediction heute
0x00400100 (beq)	Taken	Springen
0x00400204 (bne)	Not Taken	$PC + 4$

Notizen

Die CPU lernt Ihr Programm: Statistik in Flip-Flops, spekulativer Fetch auf das geratene Ziel. Unsere Tabelle zeigt die 1-Bit-Vereinfachung (letzte Aktion = Prediction). Vertiefung: in der Praxis oft 2-Bit-Saturating Counter (Zustände stark/schwach Taken bzw. Not Taken) — ein einzelner Fehlgriff kippt die Prediction nicht sofort, das dämpft Rauschen an Schleifenenden.

Spekulative Ausführung

- Herzstück moderner CPUs (Apple M, AMD Ryzen, Intel Core)
- Vorhersagen oft $> 95\%$ Trefferquote
- Fließband führt Code aus, bevor die Legalität feststeht
- Trivia: Spectre/Meltdown hängen an Spekulation und Caches

Ohne Branch Prediction würde ein 5-GHz-Prozessor durch ständiges Flushen nur einen Bruchteil leisten.

Notizen

Ohne spekulatives Raten würden tiefe Pipelines ständig stottern.

Praxis: Hazards live in Ripes

Vom Papier in den Simulator

- Ripes auf das 5-Stufen-Modell umstellen
- Bypassing-Leitungen im Datenpfad finden
- Pipeline-Bubbles bei Load-Use beobachten
- Flushing bei Schleifen erleben

Notizen

Single-Cycle beiseitelegen, 5-Stage laden: Forwarding, Stalls und Flushes in Farbe.

Ripes-Upgrade (5-Stage)

- Chip-Icon oben links
- 5-Stage Processor wählen (Standard mit Hazard/Forwarding)
- Variante ohne Forwarding: Programm liefert bei Abhängigkeiten falsche Ergebnisse (zum Ausprobieren)
- Processor-Tab: Datenpfad hat sich radikal verändert

Notizen

Für die Aufgaben: Standard-5-Stage mit Forwarding und Bubbles.

Das neue Wimmelbild

- Vier breite Pipeline-Register (IF/ID, ID/EX, EX/MEM, MEM/WB)
- Forwarding-Pfade: Linien von rechts zurück an die ALU-MUX
- Hazard Unit und Forwarding Unit als eigene Boxen



Notizen

Dicke Mauern = Pipeline-Register. ALU-MUX mit mehr Eingängen = Bypass aus MEM/WB.

Das Pipeline-Tab in Ripes

- Tab Pipeline: Zeit-Takt-Diagramm wie in den Folien
- Y-Achse: Assembler-Code
- X-Achse: Cycles
- Blöcke IF/ID/EX/MEM/WB zeigen die Lage jedes Befehls

Notizen

Perfekt, um NOPs und Stalls visuell zu jagen.

Labor-Aufgabe 1: Data Hazard provozieren

- Forwarding testen mit:

```
li    t0, 50
li    t1, 100
add   t2, t0, t1    # Ergebnis in t2
sub   a0, t2, t0     # Abhaengigkeit von t2
```

- Clock Takt für Takt: wenn add in MEM und sub in EX

Notizen

RAW herausfordern: sub greift gierig auf t2 zu, das add gerade berechnet. Hinweis: li ist ein Pseudo-Befehl und expandiert in Ripes oft zu mehreren Instruktionen — im Pipeline-Tab die echten Maschinenbefehle zählen.

Forwarding in Ripes sehen

- Im kritischen Takt zum Processor-Tab
- MUX vor dem ALU-Eingang: Forward-Steuersignal $\neq 0$?
- Maus über Bypass-Kabel: fließt dort 150 (0x96)?
- Dann haben Sie Bypassing nachgewiesen

Notizen

Forwarding Unit schaltet den MUX; Hex-Wert auf dem Rückführkabel ist Ihr ADD-Ergebnis.

Labor-Aufgabe 2: Load-Use Stall erzwingen

- Daten und Code, dann:

```
.data
array: .word 42
.text
la    s1, array
lw    t0, 0(s1)
add   a0, t0, t0
```

- Pipeline-Tab: was passiert in Takt 4 oder 5?

Notizen

Load sofort gefolgt von Nutzung — Forwarding muss scheitern, Stall greift.

Folie

80 / 85

Die NOP-Bubble identifizieren

- add weigert sich, nach EX zu rutschen
- Raster zeigt grauen/leeren Block: `nop` (Bubble)
- Ein Takt Pause für den langsamen RAM

Notizen

Decode eingefroren, NOP wandert als Platzhalter durch die hinteren Stufen.

Folie

81 / 85

Labor-Aufgabe 3: Control Hazard (Flushing)

- Endlosschleife:

```
li t0, 10
Loop:
    addi t0, t0, -1
    bne t0, zero, Loop
```

- Was passiert, wenn `bne` springt und der PC zurückgeht?

Notizen

Beobachten Sie die Befehle, die blind in IF/ID geladen wurden, bevor der Sprung feststeht.

Folie

82 / 85

Flushing in Ripes

- Vordere Stufen haben Code unter der Schleife geladen
- Ripes löscht IF/ID (durchstreichen / ungültig)
- Falsche Befehle werden zu NOP-Bubbles — Branch Penalty live

Notizen

Hazard Unit radiert IF/ID; Label-Ziel erscheint wieder in IF. Jeder Schleifendurchlauf kostet Zeit.

Labor-Aufgabe 4: Compiler-Magie selbst gemacht

- Code aus Aufgabe 2 (Load-Use Stall)
- Unabhängigen Befehl einfügen, z. B. `addi a5, a5, 1`
- Vor `lw` oder nach `add`: Stall bleibt
- Genau zwischen `lw` und `add`: Stall verschwindet

Werden Sie zum Compiler: sortieren, bis keine NOPs mehr da sind.

Notizen

Unabhängige Arbeit füllt die Bubble — gleiches Ergebnis, ein Takt schneller.

Zusammenfassung Einheit 7

- Pipelining maximiert Durchsatz, ändert Latenz eines Befehls nicht
- Fünf Stufen → viel kürzerer Takt
- Data Hazards: Forwarding/Bypassing
- Load-Use: Stall; Compiler hilft durch Scheduling
- Control Hazards: Flush; moderne Rettung Branch Prediction

Pipelining-Verständnis trennt den Coder vom Computer-Engineer.

Notizen

Forwarding, Stalling und Branch Prediction machen Hochleistungs-CPUs erst möglich.

Q&A und Hands-On

- Ripes: 5-Stage einstellen
- Hazard-Szenarien aus den Folien nachbauen
- Code darf kaputtgehen — der Simulator fängt ab
- Viel Erfolg bei der Optimierung Ihres Fließbands



Notizen

Bauen Sie bewusst RAW, Load-Use und Flush. Wenn Sie Bypässe und rote Linien verstanden haben, gehört Ihnen die Architektur.